

Week 1 - Backend AI Engineering Assignment

```
type: Assignment
track: Backend AI Engineering
week: 1
phase: Setup
status: ● Complete
tags:
  - api
  - golang
  - json
  - http
  - response
date: 2026-07-07
name: Build your first API endpoint
code: BE-01
```

Tiny Go API

Goal

Build the smallest possible backend: a server with two JSON endpoints, test it using `curl` and the browser, and publish it to a public GitHub repository.

Purpose

This assignment helps me understand the request-response loop from the server side.

Instead of only seeing the browser or client send a request, I now control the server that receives the request and sends back a response.

The backend is intentionally small so the focus stays on the basic loop:

```
Client sends request
Server receives request
Server runs the matching route
Server sends JSON response
Client displays response
```

Tech Stack

```
Language: Go
Backend library: net/http
Response format: JSON
Testing tools: curl and browser
Version control: Git
Repository hosting: GitHub
```

I used Go with the built-in `net/http` package because it allows building a simple server without installing any external framework.

Project Structure

```
tiny-go-backend/
├── .gitignore
├── go.mod
└── main.go
```

Endpoints

GET /

Returns a simple message showing that the backend is running.

Example response:

```
{
  "message": "Tiny Go backend is running"
}
```

GET /hello

Returns a greeting message from the server.

Example response:

```
{
  "message": "Hello from Go net/http!"
}
```

How the Server Works

The server uses `http.HandleFunc` to define routes.

Each route receives a request and writes a JSON response.

The helper function `sendJSON` sets the response type to JSON and encodes a Go map into JSON.

Run Locally

```
go run main.go
```

The server runs at:

```
http://localhost:3000
```

Test with curl

Test the first endpoint:

```
curl http://localhost:3000/
```

Test the second endpoint:

```
curl http://localhost:3000/hello
```

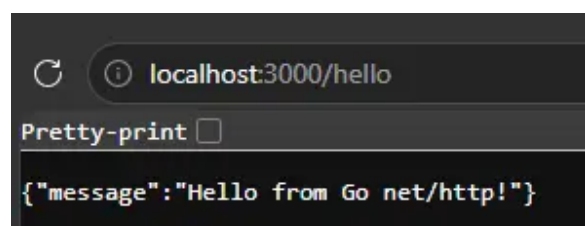
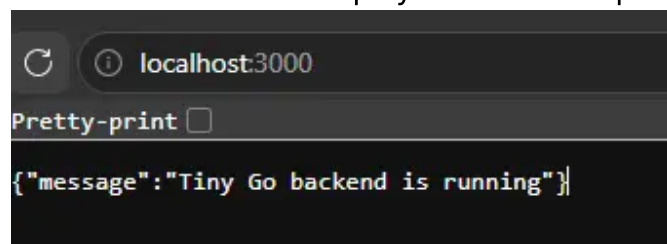
```
aya@Laptop:~$ curl http://localhost:3000/  
{"message":"Tiny Go backend is running"}  
aya@Laptop:~$ curl http://localhost:3000/hello  
{"message":"Hello from Go net/http!"}  
aya@Laptop:~$
```

Test in Browser

Open these URLs in the browser:

```
http://localhost:3000/  
http://localhost:3000/hello
```

The browser should display the JSON responses.



Request-Response Loop

When I run:

```
curl http://localhost:3000/hello
```

this happens:

```
curl sends an HTTP GET request to /hello  
Go server receives the request  
net/http matches the /hello route  
The handler creates a JSON response  
The server sends the response back  
curl displays the JSON
```

Reflection

This assignment made the request-response loop clearer because I created the server myself.

The backend does not use a database, authentication, frontend framework, or external API. It only focuses on receiving HTTP requests and returning JSON responses.

This makes it easier to understand the foundation of how web servers work before adding more complex features later.